LinTEX: Enhancing LATEXdocument accessibility for screen reader users

by

Mohammad Danyal

In partial fulfilment of the
requirements for the degree of
Bachelor of Science
in
CS408: Individual Project

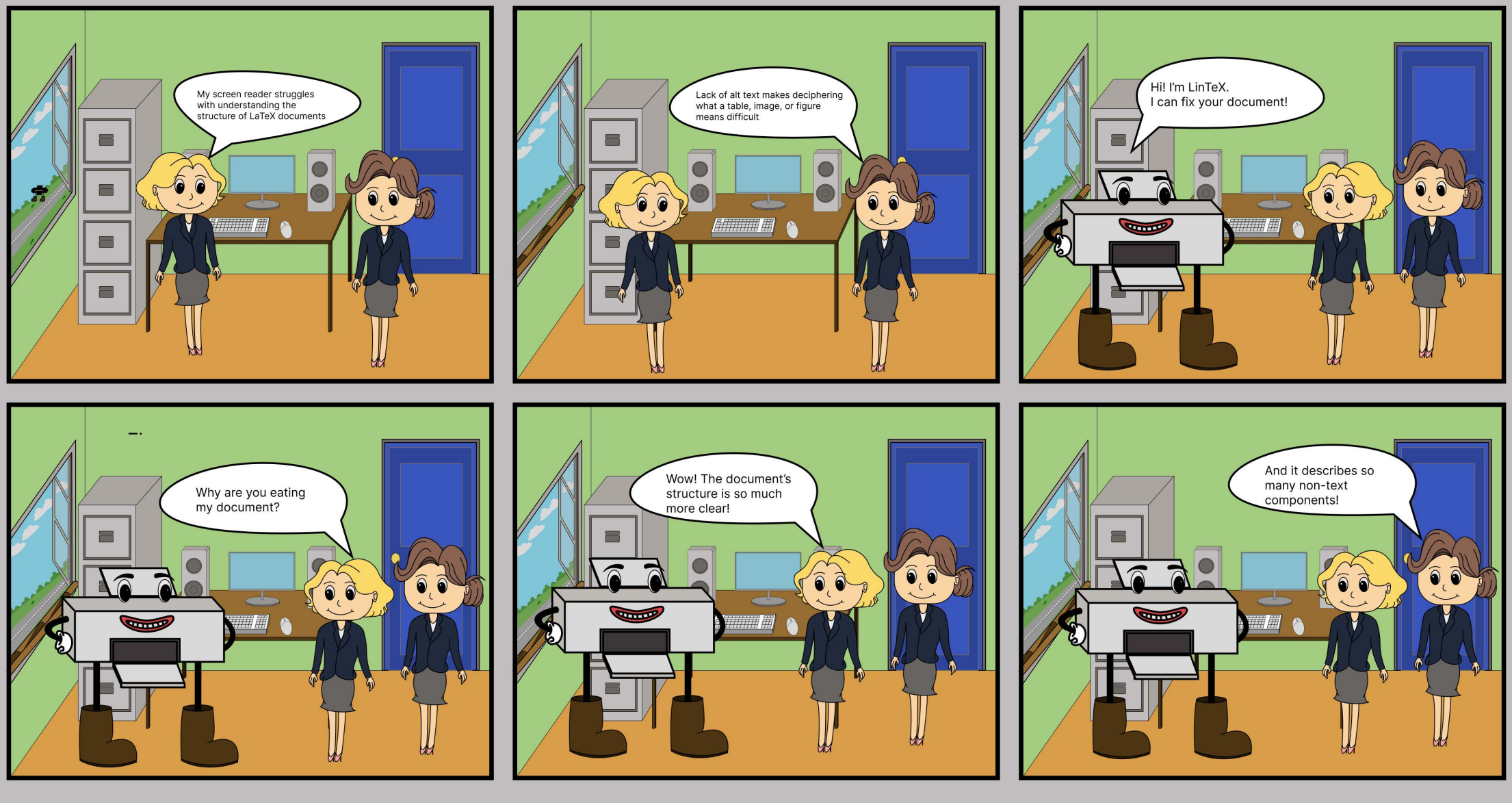
Department of
Computer and Information Sciences

April, 2026

Abstract

The abstract is a bird's eye view of the project.

It should not exceed one page. Mention the scope and objectives of the project, the methodology used,the main findings, and significance of your results.


My screen reader struggles with understanding the structure of LaTeX documents
Lack of alt text makes deciphering what a table, image, or figure means difficult
Hi! I'm LinTeX.
I can fix your document!
—.
Why are you eating my document?
Wow! The document's structure is so much more clear!
And it describes so many non-text components!

Acknowledgements

I would like to thank the participants who evaluated my project, my supervisor etc.

Contents

List of Figures

List of Tables

Glossary

accessible document	an accessible document created after using LinTEX. 12, 13
AST	abstract syntax tree 7
CGI	Common Gateway Interface 6
inaccessible document	a document that has intentionally been made inaccessible 12, 13
PEG	parsing expression grammar iii, iv, 4–7
WSL	Window's Subsystem for Linux 7

1 Introduction

1.1 Identifying LaTeX's key accessibility problems for screen reader users

Similar to a tool like Microsoft Word, LaTeXis a tool used to process documents. However, LaTeXdoes this in a programmatical manner, likening it to a programming language, whereas Microsoft Word has a that

- xetex - came after pdflatex but before lualatex created for better font handling - specify not under consideration - since latex devs focused on lualatex - was useful, but not used much - 'could be considered evolutionary deadend'/might become one/find authorative source to say it is

- introduce latex here? - rough definition ig, and purpose - (intended to be math textbook) - (small paragraph)

1.2 Why accessibility is important

- who would benefit from it - stats with disabilities e.g. how many people are blind and use screen readers - (use good sources e.g. government websites or world health organisations) - severity and frequency are considerations

1.3 Why LuaLaTeX provides better accessibility than PDFLaTeX

1.4 Creating LinTeX

- intended purpose - (this section has the finished overview diagram)

1.5 aims and objectives

1.6 Limitations of the project overall

1.7 research questions/hypothesis

This is a concise and clear overview of your dissertation(more or less 2-3 pages). You can start off the project description provided of the project that was allocated to you and flesh it out.

Include (1) the problem you have tackled, (2) why this problem is worth addressing, (3) what you did to address it - in broad terms. Detail will come later.

i.e., issue(s) on which the research will focus, shall be clearly identified and described. You shall refer to past research work relevant to the topic and objectives, i.e, of the study. You shall outline where applicable the potential research output with respect to research transfer and uptake by the community.

The introduction says: this is an overview of the project. This is why I did it (the problem) and how I tackled it. It is the *runway* into the project. It lets the marker know what to expect of your report.

If you're doing a research project, this would be the place to include the research questions you plan to address. For example:

RQ1: Where did James Bond come from?

RQ2: Why are pumpkins orange?

If you're doing a project of type 1, include a list of objectives. For example:

Objective 1: Provide software to allow James Bond to become invisible.

Objective 2: Provide software to keep track of all loyalty points in one place.

Provide a 'map' of your dissertation. For example: Section 2 reviews the background literature that was reviewed to inform this project. Then Section **??**.....

1.8 from nov progress

There is a lack of accessibility with LaTeX, largely due to usage of outdated PDFLaTeX as opposed to LuaLaTeX. Usage of this outdated TEX engine necessitates workarounds for desired features and accessibility in resulting PDF documents.

What can be done to make LaTeXdocuments more accessible?

LinTeX is a developing tool dedicated to combat this problem by focusing on assisting screen readers' abilities to read the PDF documents produced from LaTeXdocuments. It aims to cover this area through suggesting structured tagging throughout a document, suggesting

components where it may be lacking. Additionally, it will request users to input appropriate alternative text ('alt text') for non-text components of the document, including figures, images, and tables. Further focus may be placed on features such as ensuring appropriate contrast in any colours in the document, appropriate Sans Serif fonts, and flexible document design, largely depending on what development time allows for.

Additionally, as a tool focused on promoting accessibility, it too should be accessible. Therefore, the tool will ensure strict adherence to the Web Content Accessibility Guidelines 2 (WCAG 2)

Why should I use LinTeX?

While the tool is not limited strictly to the field, it is aimed to assist the many academics that find difficulties in accessing reports and academic literatures. By utilising this tool, you can ensure a wider demographic of users can view your documents, without necessitating an especially large amount of research in accessibility; it has been done for you.

2 Background Literature

This section should provide the grounding for your project. You will be able to refer back to this in Section **??**, where you report on design decisions and methodology.

There should be a strong link to your research questions (for project types 2 and 3). There will be a strong link to the software design decisions for project type 1.

Don't just include a paragraph for each paper you read. Synthesize it and create a story line.

Review between 5 and 10 authoritative sources. For research based projects, some of these should be from peer-reviewed journals or conferences.

This section will reference literature like this **turing1950computing**. Sloppy referencing will (1) take time and effort, and (2) lose you marks if you don't do it.

2.1 (brief intro into background, such as accessibility or latex-related background)

2.2 Choosing a programming language

2.3 Choosing a PEG library under Go programming language

2.4 Choice of libraries

- net/http - path/filepath (effectively the only option tbf) - html/template (effectively the only option tbf, outside of raw print?) - os & os/exec

2.5 guidelines for accessibility

- guidelines should be understood by me - languages guidelines are written in - check if authority publishiing guidelines is a good source i.e. is an international standard instead of a random person

2.6 Choosing a software development methodology

2.7 Choosing a screen reader

2.8 User study design choices

2.9 Designing a GUI

- Tailwind choice? - Flowbite choice? - Potential HTMX usage if time allowed for it?

2.10 from nov progress

2.11 Programming Language Comparisons

There were many considerations when deciding what programming language to write LinTEX in. Languages chosen were based on those that I had prior experience in, to ensure minimal hinderance from the language itself to the development of the project. An important factor is ensuring LinTEX's maintainability in the future. Therefore, only the languages which have actively maintained parsing expression grammar (PEG) libraries will be considered, as can be seen in Table 2. The most recent update times provided in Table 2 are in reference to 24th November 2025.

Table 2: Languages and related PEG frameworks

Language	PEG library link	PEG library is in development
Python	https://pythonhosted.org/pyPEG2/	No, last updated in October 2015
Python	https://github.com/erikrose/parsimonious	Yes, last updated in November 2025
JavaScript	https://github.com/pegjs/pegjs	No, last updated in November 2019
JavaScript	https://peggyjs.org/	Yes, last updated in September 2025
Go	https://github.com/pointlander/peg	Yes, last updated in November 2025
Go	https://github.com/mna/pigeon	Yes, last updated in November 2025
C	N/A	N/A
Java	https://github.com/sirthias/parboiled	Yes, last updated in November 2025
C#	https://code.google.com/archive/p/peg-sharp/	No, last updated in May 2011
C#	https://github.com/otac0n/Pegasus	No, last updated in December 2023
C++	https://github.com/yhirose/cpp-peglib	Yes, last updated in July 2025
PHP	https://github.com/hafriedlander/php-peg	No, last updated in May 2014
TypeScript	https://github.com/metadevpro/ts-pegjs	No, last updated in November 2024
TypeScript	https://github.com/EoinDavey/tsPEG	Yes, last updated in November 2025
Dart	https://github.com/darmie/dart-peg	No, last updated in July 2015

Based on the current criteria, we can rule out C, C#, PHP, and Dart.

As many languages remain, there is a need for a stricter criteria: suitability to be utilised as

a web application as well as compatibility with the University of Strathclyde's development web server (devweb) system.

Table 3: Languages and related website development support

Language	Web development support	DevWeb support
Python	Yes	Yes
JavaScript	Yes	Yes
Go	Yes	No
C	No, except for rare exceptions	No
Java	Yes, but not generally used	Yes
C#	Yes, but not generally used	No
C++	Yes, but not generally used	No
PHP	Yes	Yes
TypeScript	Yes	Yes
Dart	Yes	Yes

It should be noted that while Go, C, C#, and C++ are not directly compatible with devweb, as referenced in Table 3, they can be supported through use of Common Gateway Interface (CGI) scripts, and consequently should not be fully disregarded for this alone. However, based on their support for web development, C, Java, C#, and C++ will have a lower priority for being a chosen language.

This leaves us with a focus placed on JavaScript, Go, and TypeScript. However, JavaScript will be removed from consideration as a strictly typed language is preferred for our context. Therefore, leaving Go and TypeScript as the main considerations for this project. Finally, as I have more experience and a stronger preference towards Go as opposed to TypeScript, it was the chosen programming language for the project.

2.12 Library Comparison

Go has two libraries that are used in relation to PEG systems: Pigeon by Martin Angers (mna), and PEG by Andrew Snodgrass (pointlander). Direct comparisons with core components of both have been made in Table 4.

While examples are relatively similar, the simplicity of installation for Pigeon as opposed to PEG, combined with the depths of documentation both in generated code and outwith it, makes Pigeon a more ideal option. Additionally, its greater number of features gives LinTeX more to make use of. Therefore, Pigeon was the chosen library for the project.

Table 4: Comparison between PEG and Pigeon

	Pigeon	PEG
Installation	Requires running a `go install` command and setting up environment variables	Requires supporting bash scripting (through Window's Subsystem for Linux (WSL) for Windows users), as well as running a `go install` command, cloning the repository, and running `go generate && go build`
Documentation	Contains a Godoc page, wiki page, an examples subdirectory, and a detailed README.md providing what PEG features are supported, command line interface usage, installation, and examples	Contains a detailed README.md and a PEG file syntax file containing what PEG features are supported, installation steps, and example projects links
Generated Code	Generated Go code can be used to configure settings on the parser, view statistics or error information, debug, enable or disable memoization, and parse readers, files, and strings. Additionally, the generated code is well documented, with several comments surrounding usage of public functions	Generated Go code can be used to run the PEG script on a provided string, debug and output the abstract syntax tree (AST), output errors in parsing, and enable or disable memoization
Examples	Provides three direct examples: detecting indentation in files, parsing JSON files, and a calculator example. Examples were kept short (in regards to PEG scripts) with the calculator and indentation examples showcasing usage via a main function	Provides two project links for examples on PEG usage: a natural date/time parser and a scientific names parser. Both projects have large PEG scripts, with most lines being simple to read. The natural date/time project was intuitive to use and understand program flow

2.13 Design Focus

LinTeX will take strong inspiration from Integrated Development Environments (IDEs) with regards to its User Interface/User Experience (UI/UX) design. This is because LaTeX in structure, is similar to a programming language, even being Turing complete **latexturingcomplete**. Hence, an environment suitable to programming is suitable to LaTeX, with some special considerations put aside for its quirks. For example, depending on the user's screen size and preferences, they may wish to have the output of the LaTeX documents taking up half of the screen, a separate browser window or monitor, or out of the way until compilation.

3 Specification & Design

Describe all details of the design and procedures used to achieve the project objectives. Do this chronologically.

It should be detailed enough to allow for an assessment of the rigour of your process, and, in the case of research projects, in terms of how well grounded your research is in the research literature. In these cases, refer back to relevant sections in the previous chapter.

Say which software lifecycle approach you used e.g., Waterfall, Spiral, Agile.

How did you gather user requirements?

3.1 Methodology

Which methodology did you choose?

3.2 Analysis

How did you decide on the particular software artifact you decided to develop?

3.3 Requirements

Here you explain what the functional and non-functional requirements are. Explain how you prioritised them. See `https://www.nuclino.com/articles/functional-requirements` for more information.

Functional requirement: "The system must ***do*** [requirement]."

Non-functional requirement: "The system shall ***be*** [requirement]."

Well-written functional requirements typically have the following characteristics:

Necessary: Although functional requirements may have different priority, every one of them needs to relate to a particular business goal or user requirement.

Concise: Use simple and easy-to-understand language without any unnecessary jargon to prevent confusion or misinterpretations.

Attainable: All requirements you include need to be realistic within the time and budget constraints set in the business requirements document.

Granular: Do not try to combine many requirements within one. The more precise and granular your requirements are, the easier it is to manage them.

Consistent: Make sure the requirements do not contradict each other and use consistent terminology.

Verifiable: It should be possible to determine whether the requirement has been met at the end of the project.

3.3.1 Functional Requirements

This is the **WHAT** of your artifact.

Functional requirements are product features that developers must implement to enable the users to achieve their goals. They define the basic system behavior under specific conditions.

Functional requirements need to be clear, simple, and unambiguous.Examples:

- The system must send a confirmation email whenever an order is placed.
- The system must allow blog visitors to sign up for the newsletter by leaving their email.
- The system must allow users to verify their accounts using their phone number.

3.3.2 Non-Functional Requirements

This is the **HOW** of your artifact. Example non-functional requirement: "When the submit button is pressed, the confirmation screen must load within 2 seconds."

3.4 Design

3.4.1 Interface Design

Explain how you used wireframes, and how you tested these to design the user interface.

3.4.2 System Design

Show how you designed your database (if appropriate) and how you designed your system architecture, and the individual parts. Use UML and an Entity Relationship diagram

4 Implementation

4.1 general overview of technologies used

4.2 website with server-client relationship

4.3 possibly include things about peg

4.4 verification & validation

- (manual testing and error messages for server-side errors that should not occur in most cases)

5 User study

A user study is an approach taken to understand the requirements of users in relation to your application (**user_study_definition**). It can be used to better understand user requirements for those who will be using the system and consequently improve design decisions.

5.1 Study design

The user study was designed with the goal of evaluating the effectiveness of the tool, LinTEX, and identifying areas of improvement for the tool. To meet these goals, the user study was split into six key stages:

1. An introductory stage where the goal is to identify where the participant's knowledge lies in related topics.
2. An initial listening of the inaccessible document, which was produced for this user study.
3. A questioning stage based on the initial listening of the inaccessible document.
4. A think-aloud session for understanding the accessibility of LinTEX.
5. A final listening of the newly created accessible document by using LinTeX.
6. A questioning stage based on the final listening of the accessible document.

These stages allowed for a comparison of the before and after of using LinTEX, through stages 2 and 5, as well as associated questioning stages 3 and 6, allowing both qualitative and quantitative data to be obtained. Additionally, the think-aloud session provides an opportunity to judge the usability of LinTEX.

Finer details for the user study can be found in the user study script (section A.4), with the reasoning behind their choices found in the user study design choices (section 2.8).

5.2 Hypothesis

There were several hypotheses that this user study aimed to prove or disprove:

H1 The accessible document created by LinTEX showed an improvement from the inaccessible document initially used in terms of the documents' accessibility for screen reader users.

H2 LinTEX is reasonably accessible.

H3 LinTEX provided useful error messages.

H4 The inaccessible document used in the initial listening is not memorable.

5.3 Results

The sample size for this user study was small, with 5 participants being recruited for it.

5.3.1 Memorable document extracts

The following table, namely Table 5, showcases the results from the question in the initial questioning (stage 3) that requests participants to summarise the document.

Table 5: Memorable aspects of document extracts

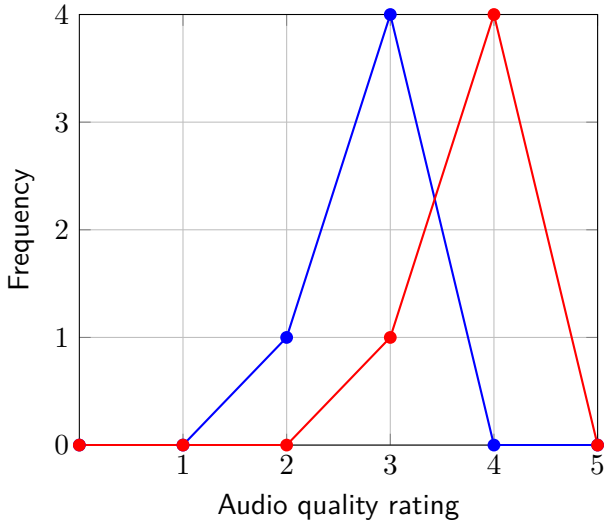
Document extract	Comment
Improving spoken skills	The majority of participants recalled spoken skills being improved through tongue twisters, yet there was minimal mentions of other techniques referenced in the inaccessible document.
House sale	A little over half of the participants had recalled a house being mentioned, with the currency and specific pricing often incorrect.
Favourite pet	All users recalled the extract about the cat being the favourite pet.
Muhammad and Noah name comparisons in Scotland	All users recalled this extract, with finer details such as: whose name was more popular, the positions of the names, and what names were in the context were often incorrect.
Grade comparisons	All users recalled this extract, with finer details such as: the friends' names, John's name, the class codes, whether the extract referenced John's friends or general students, who was assigned what grades, and the grades themselves, were often incorrect.

It should be noted that participants that had requested to pause or repeat a segment of the initial listening showcased a better recollection of the document extracts, with the average participant able to recall 4 extracts on average.

5.3.2 Accessible document and inaccessible document comparison

The following figure, namely Figure 1, showcases the comparison of Likert scale data obtained from the user study that queries the participants about the quality of the screen reader's audio output in regards to its contents. This data was used to create a frequency polygon graph, of which the blue line showcases the data from the initial questioning (stage 3), and the red line showcases the data from the final questioning (stage 6).

Figure 1: Comparing screen reader output quality ratings before and after applying LinTEX



The key difficult components of both the initial listening (stage 2) and final listening (stage 5) can be found in Table 6. It should be noted that this is not an exhaustive listing of all difficult components from both stages, but rather the key points that a majority of participants in the user study had placed emphasis on. For example, a few participants had noted that the scale of numbers included in a table had an impact on its legibility.

Table 6: Difficult aspects from the initial and final listenings

Initial document	Final document
Repetition of 'percent' in the second table was found to be distracting.	Repetition of 'percent' in the second table was found to be distracting.
Images were difficult to keep track of when and where they had occurred.	Tables contained too much metadata, with focus placed on 'column N' being repeated for each table cell.
Images being plain file names made it difficult to ascertain their purpose.	Document metadata could be confused with the contents of the document, with emphasis placed on column numbers being mixed with name positions in the first table.
Headings and pages did not have sufficiently long pauses before beginning.	
Tables were read too fast with no pauses between table cells.	
The table containing names is viewed from top to bottom, whereas the screen reader read from left to right.	

The key simple components of both the initial listening (stage 2) and final listening (stage 5) can be found in Table 7. It should be noted that this is not an exhaustive listing of all simple components from both stages, but rather the key points that a majority of participants in the user study had placed emphasis on. For example, many participants had enjoyed the inclusion of an example tongue twister in place of the relevant image.

Table 7: Simple aspects from the initial and final listenings

Initial document	Final document
Standalone sentences were read slower and with pauses. Sentences separated by images or tables were easier to distinguish.	Standalone sentences were read slower and with pauses. Sentences separated by images or tables were easier to distinguish.
Lists provided in bullet point form were simple to understand due to a clear starting point.	Lists provided in bullet point form were simple to understand due to a clear starting point. Each point in the list started with 'bullet' instead of 'bullet point' which was the case previously, consequently speeding up the screen reader's output.
The table discussing grades is read left to right, whereas the table discussing names is read top to bottom. When the table can be viewed left to right, it followed the same order the screen reader used when creating the audio output.	The table discussing grades is read left to right, whereas the table discussing names is read top to bottom. When the table can be viewed left to right, it followed the same order the screen reader used when creating the audio output.
Supplementary sentences included before or after images and tables improved the overall clarity of the related extracts.	Supplementary sentences included before or after images and tables improved the overall clarity of the related extracts.
	Images were easier to understand due to their included descriptors.
	The screen reader included more pauses and spoke slower, with emphasis placed on tables.
	Structural indicators such as 'out of list', 'out of table', and 'table with M columns and N rows' improved tracking of where the screen reader was on the document at that moment.
	Reading the table heading before each table data cell improved tracaking of what data correlated to what name in the first table.

It should be noted that all participants required less pausing or repetition of an extract in the final listening (stage 5) compared to the initial listening (stage 2).

5.3.3 Likelihood to support screen reader users

The following figure, namely Figure 2, showcases the results from the Likert scale data obtained in the final questioning (stage 6) that queries participants on their likelihood to support screen reader users in the future.

5.3.4 LinTEX usability

The following table, namely Table 8, highlights areas of which usability could be improved or were found largely useful during the think-aloud session (stage 4).

Figure 2: Participants likelihood to support screen reader users in the future

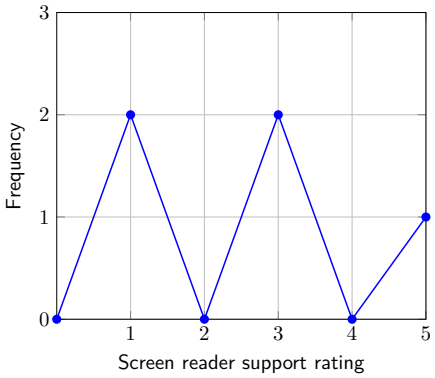


Table 8: LinTEX usability

Affected area	Comment
Uploading a file	Lacked responsiveness in regards to actions performed when the upload button was clicked without a selected file and informing the participant a file has been uploaded.
Button phrasing	Phrasing of 'generate output' button caused participants to assume it could only be clicked after all errors were fixed.
Scrolling options	Scrolling options for the in-built code editor, errors, and the website as a whole, made navigation more difficult.
Submitting a document	A keybind to generate the document's PDF output was requested.
Error extracts	Most error extracts, the code taken from where the error had occurred, were helpful. However, syntax errors such as a missing ending curly braces only provided the starting curly braces within its extract. It should be noted that many users utisilised the browser's search features to locate where the error had occurred.
Image error messages	No difficulties found.
Table error messages	Attempts were made to place the 'tabular' group within the required argument of '\tagpdfsetup'.
Tagging error messages	Uncertainty existed with what should be provided to the required argument of '\DocumentMetadata' and how to handle several key-value pairs being provided to it, with some attempts to add an extra required argument.
Error messages	Examples helped to achieve the correct output, with long error messages found from requesting more details to have alleviated many prior concerns.
Text size	Text size was small.
Colour scheme	The colour scheme of the website may be inaccessible for users with dyslexia.

5.4 Analysis

A lot of filler text.

A lot of filler text.

A lot of filler text.

A lot of filler text.

A lot of filler text.

A lot of filler text.

A lot of filler text. A lot of filler text.

A lot of filler text.

A lot of filler text.

A lot of filler text.

A lot of filler text.

A lot of filler text.

A lot of filler text. A lot of filler text.

A lot of filler text.

A lot of filler text.

A lot of filler text.

A lot of filler text.

A lot of filler text.

A lot of filler text. A lot of filler text.

A lot of filler text.

A lot of filler text.

A lot of filler text.

A lot of filler text.

A lot of filler text.

A lot of filler text.

5.5 Hypothesis testing

A lot of filler text.

A lot of filler text.

A lot of filler text.

A lot of filler text.

A lot of filler text.

A lot of filler text.

6 Discussion and reflection

The conclusion is similar to when a plane lands. You don't rewrite the introduction. You say something like - I addressed the problem outlined in the introduction, and I built some software to do this. I tested the software like this ADD FEW WORDS.

Summarize main findings drawn from the project work. Mention the objectives or research questions. Do not repeat points raised in the discussion and reflection Section. If applicable, you can make recommendations. The conclusion should NOT contain any references.

7 Conclusion

7.1 Project summary

7.2 Project limitations

7.3 Achievements – potentially optional/swap with final thoughts or another area?

7.4 Future work

7.5 Reflection

7.6 Final thoughts

The conclusion is similar to when a plane lands. You don't rewrite the introduction. You say something like - I addressed the problem outlined in the introduction, and I built some software to do this. I tested the software like this ADD FEW WORDS.

Summarize main findings drawn from the project work. Mention the objectives or research questions. Do not repeat points raised in the discussion and reflection Section. If applicable, you can make recommendations. The conclusion should NOT contain any references.

A Appendix

This is where you can include your documentation.

Remember that the marker is not required to read this, but might well check to ensure that you have included product documentation, and ethical approval, as required.

A.1 Ethical approval form

If your project required you to do any evaluation with humans, you MUST include this. It can be downloaded from the Ethics system.

`https://local.cis.strath.ac.uk/wp/extras/ethics/index.php`

A.2 Participant information sheet

If your project required you to do any evaluation with humans, you MUST include this

`https://www.strath.ac.uk/ethics/informationsheetandconsentform/`

A.3 Consent form

If your project required you to do any evaluation with humans, you MUST include this.

`https://www.strath.ac.uk/ethics/informationsheetandconsentform/`

A.4 User study script

something